

Book a Doctor – Project Documentation

1. Introduction

Project Title: Book a Doctor – Full-Stack Healthcare Appointment Booking Platform

Team Members:

Name	Role
Barnala Ram Mohan	Team Lead
Kokkiligadda Sasidhar	Member – Developer (Project Implementation)
Kusampudi Jahnavi	Member
POLIMETLA BINNI	Member
Sodadasi Nissy Joice	Member

2. Project Overview

Purpose

Book a Doctor is a full-stack healthcare appointment booking platform that connects patients with verified doctors online. The goal is to simplify the traditional, often manual process of finding a doctor and scheduling a consultation by providing a single web application where:

- Patients can discover doctors by specialisation, book appointment slots, and track the status of their requests.
- Practitioners can apply to become verified doctors on the platform, manage their availability, and review incoming appointment requests.
- Administrators can vet and approve doctor applications, and oversee all users and appointments from a central dashboard.

The system is built on the MERN stack (MongoDB, Express.js, React, Node.js) using an MVC architecture on the backend, giving it a clear separation between data models, business logic, and API routes.

Key Features

- **Role-based access** — three distinct user roles (Patient, Doctor, Admin), each with its own dashboard and permissions, enforced via JWT authentication and middleware guards.
- **Secure authentication** — user registration and login with bcryptjs password hashing and JWT-based session tokens.
- **Doctor application workflow** — any registered patient can apply to become a doctor; the application stays in a "pending" state until an admin approves or rejects it.
- **Admin approval dashboard** — admins can approve/reject doctor applications, create doctor accounts directly, and remove users or doctors from the platform.
- **Appointment booking** — patients can browse approved doctors and book a specific date/time slot, with built-in double-booking prevention for the same doctor, date, and time.
- **Document upload** — patients can attach a supporting medical document (via multer) when booking an appointment.
- **Appointment lifecycle management** — appointments move through pending → approved/rejected → completed status, controlled by the doctor handling the request.
- **In-app notifications** — users receive real-time-style notifications (new appointment requests, status changes, doctor approval decisions) stored on their profile, with options to mark as read or clear them.
- **Responsive UI** — built with React 18, Redux Toolkit for state management, React Router for navigation, and Material UI/Bootstrap for styling.

3. Architecture

3.1 Frontend (React)

The client is a **React 18 + Vite** single-page application.

- **Routing** — react-router-dom (BrowserRouter) defines all routes in App.jsx. Routes are grouped into Public (/, /login, /register), Patient (/dashboard, /apply-doctor, /appointments, /notifications), Doctor (/doctor/dashboard, /doctor/profile, /doctor/appointments), and Admin (/admin/dashboard, /admin/users, /admin/doctors, /admin/appointments) routes.
- **Route protection** — a ProtectedRoute component checks for a JWT token in localStorage and redirects unauthenticated users to /login; an adminOnly flag further restricts admin routes to users whose profile has isAdmin: true.
- **State management** — @reduxjs/toolkit powers a small global store (redux/store.js) with two slices:
 - userSlice — holds the logged-in user's profile.

- alertSlice — holds a global loading flag used to show/hide a Spinner during API calls.
- **API communication** — a single pre-configured Axios instance (api/axiosInstance.js) reads the backend base URL from VITE_API_BASE_URL (client .env), automatically attaches the JWT as a Bearer token to every request via a request interceptor, and globally handles 401 responses by clearing the token and redirecting to /login.
- **Session rehydration** — on app load, App.jsx checks for a stored token and calls GET /user/getUserInfo to restore the logged-in user into Redux state.
- **Pages** — organized by role under src/pages/ (user/, doctor/, admin/), plus shared pages (Home, Login, Register) and shared components (Layout, ProtectedRoute, Spinner).
- **UI libraries** — Material UI (@mui/material, @mui/icons-material), Bootstrap, and react-toastify for toast notifications.

3.2 Backend (Node.js + Express.js)

The server follows the **MVC (Model–View–Controller)** pattern, entry point server.js:

- **Express app setup** — configures CORS (allowing localhost, any *.onrender.com subdomain, and the FRONTEND_URL env variable), JSON/urlencoded body parsing, and morgan request logging.
- **Static file serving** — /uploads is served statically so uploaded appointment documents can be accessed by URL.
- **Routing layer** (routes/) — five route modules mounted under /api:
 - /api/auth → authRoutes.js (register, login)
 - /api/user → userRoutes.js (profile, apply-as-doctor, notifications)
 - /api/doctor → doctorRoutes.js (doctor profile & appointment management)
 - /api/appointment → appointmentRoutes.js (booking, viewing, cancelling)
 - /api/admin → adminRoutes.js (user/doctor/appointment administration)
- **Controller layer** (controllers/) — contains the business logic for each route module (authController, userController, doctorController, appointmentController, adminController), each function wrapped in try/catch with consistent { success, message, data } JSON responses.
- **Middleware layer** (middlewares/authMiddleware.js):
 - authMiddleware — verifies the JWT from the Authorization: Bearer <token> header and attaches the decoded user ID to req.userId.
 - adminMiddleware — runs after authMiddleware; looks up the user and rejects the request unless isAdmin is true.
- **File uploads** — multer handles the medical document uploaded during appointment booking, storing files under server/uploads/.
- **Startup sequence** — on boot, the server connects to MongoDB (config/db.js) and then runs config/seedAdmin.js to ensure a default admin account exists.
- **Error handling** — a global 404 handler for unmatched routes and a global error-handling middleware that returns a JSON 500 response.

3.3 Database (MongoDB)

The database is **MongoDB**, accessed through **Mongoose** schemas (server/models/). There are three collections:

User (userModel.js)

Field	Type	Notes
name	String	required
email	String	required, unique, lowercase
password	String	required, hashed with bcrypt, min length 6
phone	String	optional
isDoctor	Boolean	default false
isAdmin	Boolean	default false
notifications	Array	unread notification objects
seenNotifications	Array	read notification objects
timestamps	—	createdAt / updatedAt auto-managed

Doctor (doctorModel.js)

Field	Type	Notes
userId	String	references the linked User_id
fullname, email, phone, address	String	required
specialisation	String	required
experience	String	required
fees	Number	required
timings	Object	availability window (e.g. start/end time)
status	String	pending approved rejected, default pending

Appointment (appointmentModel.js)

Field	Type	Notes
userId	String	references the booking patient
doctorId	String	references the Doctor._id
doctorInfo	Object	snapshot of doctor details at booking time
userInfo	Object	snapshot of patient details at booking time
date	String	required, format DD-MM-YYYY
time	String	required
document	String	path to uploaded supporting document, optional
status	String	pending approved rejected completed, default pending

Relationships & interaction pattern:

- Doctor.userId and Appointment.userId / Appointment.doctorId are stored as plain strings rather than Mongoose ObjectId references, so relationships are resolved manually in the controllers (e.g. looking up a User by the ID stored on a Doctor document) rather than via .populate().
- Appointment intentionally denormalizes doctor/patient info (doctorInfo, userInfo) at the time of booking, so the appointment record remains meaningful even if the doctor's or patient's profile changes later.
- Notifications are embedded directly inside the User document (notifications / seenNotifications arrays) rather than stored in a separate collection, keeping notification reads/writes to a single document update.

4. Setup Instructions

4.1 Prerequisites

Make sure the following are installed before setting up the project:

Software	Recommended Version	Purpose
Node.js	v18 or later	Runs the Express backend and the Vite dev server
npm	v9+ (bundled with Node.js)	Installs dependencies for both client and server
MongoDB	Local install or a MongoDB Atlas cluster	Application database
Git	Latest	Cloning the repository
A code editor	e.g. VS Code	Editing the project

4.2 Installation

1. Clone the repository

2. Install backend dependencies

3. Install frontend dependencies

4. Configure environment variables

Create a `.env` file inside `server/` with the following keys:

Create a `.env` file inside `client/` with the following key:

 **Security note:** Never commit real `.env` values (database credentials, JWT secret, admin password) to version control. Add `.env` to `.gitignore` in both `client/` and `server/`, and rotate any credentials that may have been exposed previously.

5. Run the backend

The API will start on `http://localhost:5000`. On first run, it connects to MongoDB and automatically seeds the admin account defined in `.env`.

6. Run the frontend

In a separate terminal:

The app will be available at `http://localhost:3000` (or the port Vite reports).

7. Log in

- As **Admin**: use the `ADMIN_EMAIL` / `ADMIN_PASSWORD` from the server `.env`.

- As **Patient**: register a new account from the /register page.
- As **Doctor**: register as a patient, then apply via "Apply for Doctor" — an admin must approve the application before doctor features unlock.

5. Folder Structure

5.1 Client (client/)

5.2 Server (server/)

Key organizational principle: the server strictly follows MVC — routes only wire up URLs to controller functions and apply middleware; controllers contain all business logic and talk to models; models define the MongoDB schema and validation rules. This keeps each layer independently testable and easy to navigate.

6. Running the Application

Note: Based on the actual package.json scripts in this project, the frontend does not define an npm start command — Vite projects use npm run dev instead. The commands below reflect what will actually work; the backend does support npm start.

Backend — from the server/ directory:

This runs node server.js, which connects to MongoDB, seeds the admin account if it doesn't exist, and starts the Express API on http://localhost:5000 (or the PORT set in .env).

(For development with auto-restart on file changes, use npm run dev instead, which runs the server via nodemon.)

Frontend — from the client/ directory, in a separate terminal:

This starts the Vite development server, available by default at http://localhost:3000 (or the port Vite reports in the terminal).

Build for production (frontend):

Once both servers are running, open the frontend URL in your browser — it will communicate with the backend via the VITE_API_BASE_URL configured in client/.env.

7. API Documentation

All endpoints are prefixed with the base URL, e.g. `http://localhost:5000/api`. Protected endpoints require an `Authorization: Bearer <token>` header. Every response follows the shape `{ success, message, data }` (fields vary slightly by endpoint).

7.1 Auth Routes — `/api/auth`

Method	Endpoint	Access	Description
POST	<code>/register</code>	Public	Register a new patient account
POST	<code>/login</code>	Public	Log in and receive a JWT token

POST `/api/auth/register` Request body:

Example response (201):

POST `/api/auth/login` Request body:

Example response (200):

7.2 User Routes — `/api/user` (all require auth)

Method	Endpoint	Description
GET	<code>/getUserInfo</code>	Get the logged-in user's profile
POST	<code>/applyDoctor</code>	Submit an application to become a doctor
GET	<code>/getAllDoctors</code>	List all approved doctors
GET	<code>/getNotifications</code>	Get the user's notifications
POST	<code>/markAllRead</code>	Move all unread notifications to "seen"
DELETE	<code>/deleteAllNotifications</code>	Clear all notifications

POST `/api/user/applyDoctor` Request body:

Example response (201):

7.3 Doctor Routes – /api/doctor (all require auth)

Method	Endpoint	Description
GET	/getDoctorInfo/:userId	Get a doctor's profile by their linked user ID
POST	/updateProfile	Update the logged-in doctor's own profile
GET	/getDoctorAppointments	Get appointments assigned to the logged-in doctor
POST	/updateAppointmentStatus	Approve/reject/complete an appointment

POST /api/doctor/updateAppointmentStatus Request body:

Example response (200):

7.4 Appointment Routes – /api/appointment (all require auth)

Method	Endpoint	Description
POST	/bookAppointment	Book an appointment (supports optional file upload field document, max 5MB, image/PDF/DOC)
GET	/getUserAppointments	Get the logged-in patient's appointments
DELETE	/cancelAppointment/:id	Cancel an appointment (only the owning patient may cancel)

POST /api/appointment/bookAppointment (*multipart/form-data*) Fields:

Example response (201):

Example error response (400) – slot already taken:

7.5 Admin Routes – /api/admin (all require auth + admin role)

Method	Endpoint	Description
GET	/getAllUsers	List all non-admin users
GET	/getAllDoctors	List all doctors (any status)
GET	/getAllAppointments	List all appointments platform-wide
PUT	/changeDoctorStatus	Approve or reject a doctor application
POST	/createDoctor	Directly create a pre-approved doctor account
DELETE	/deleteDoctor/:id	Delete a doctor profile and its linked user account
DELETE	/deleteUser/:id	Delete a patient account

PUT /api/admin/changeDoctorStatus Request body:

Example response (200):

POST /api/admin/createDoctor Request body:

Example response (201):

7.6 Common Error Responses

Status	Meaning	Example
400	Bad request / validation error	{ "success": false, "message": "Email already registered." }
401	Missing/invalid/expired token	{ "success": false, "message": "Invalid or expired token." }
403	Forbidden (e.g. non-admin hitting an admin route)	{ "success": false, "message": "Access denied. Admins only." }
404	Resource not found	{ "success": false, "message": "User not found." }
500	Server error	{ "success": false, "message": "Server error." }

8. Authentication & Authorization

The application uses **stateless, JWT-based authentication** — there are no server-side sessions; every request carries its own proof of identity.

8.1 Registration & Login

- **Registration** (POST /api/auth/register) — the password is hashed with bcryptjs (bcrypt.genSalt(10) + bcrypt.hash) before being stored; the plain-text password is never saved to the database.
- **Login** (POST /api/auth/login) — the submitted password is compared against the stored hash with bcrypt.compare(). On success, the server issues a **JSON Web Token (JWT)** signed with JWT_SECRET (from server/.env), containing the user's MongoDB _id as its payload, with an expiry set by JWT_EXPIRES_IN (default 7d).

8.2 Token Storage & Usage (Client Side)

- The JWT returned at login is stored in the browser's localStorage under the key token.
- The Axios instance (api/axiosInstance.js) uses a **request interceptor** to automatically attach the token to every outgoing API call as an Authorization: Bearer <token> header — individual pages/components never have to attach it manually.
- A **response interceptor** watches for 401 Unauthorized responses; if one occurs (token missing, invalid, or expired), it clears the token from localStorage and redirects the user to /login.

- On app load, App.jsx checks for an existing token and, if present, calls GET /api/user/getUserInfo to restore the user's profile into Redux state — this is what keeps a user "logged in" across page refreshes without a traditional server session.

8.3 Verifying Requests (Server Side)

- authMiddleware (middlewares/authMiddleware.js) runs on every protected route. It reads the Authorization header, verifies the token's signature and expiry with jwt.verify(token, JWT_SECRET), and attaches the decoded user ID to req.userId for the controller to use. Requests with a missing, malformed, invalid, or expired token are rejected with 401.
- adminMiddleware runs **after** authMiddleware on admin-only routes. It looks up the full user record by req.userId and checks the isAdmin flag, rejecting the request with 403 Forbidden if the user isn't an admin.

8.4 Authorization Model (Role-Based Access Control)

Authorization is role-based, driven by two boolean flags on the User model: isDoctor and isAdmin (a user with both false is a plain patient).

Role	How it's granted	What it can access
Patient	Default for every new registration	Book/view/cancel own appointments, apply to become a doctor, manage own notifications
Doctor	isDoctor set to true once an admin approves their doctor application (or when an admin creates the account directly via createDoctor)	Everything a patient can do, plus: update own doctor profile, view/manage appointments assigned to them
Admin	isAdmin set to true — seeded automatically on server startup from ADMIN_EMAIL/ADMIN_PASSWORD in .env (config/seedAdmin.js)	Full platform access: manage users, approve/reject/create doctors, view all appointments

On the frontend, ProtectedRoute mirrors this model: routes wrapped without any flag just require a valid token (logged in), while routes wrapped with adminOnly additionally check

user.isAdmin from Redux state and redirect non-admins back to /dashboard. Note this is a **UI-level convenience only** — the real enforcement happens on the backend via authMiddleware/adminMiddleware, since client-side checks can always be bypassed.

8.5 Security Notes

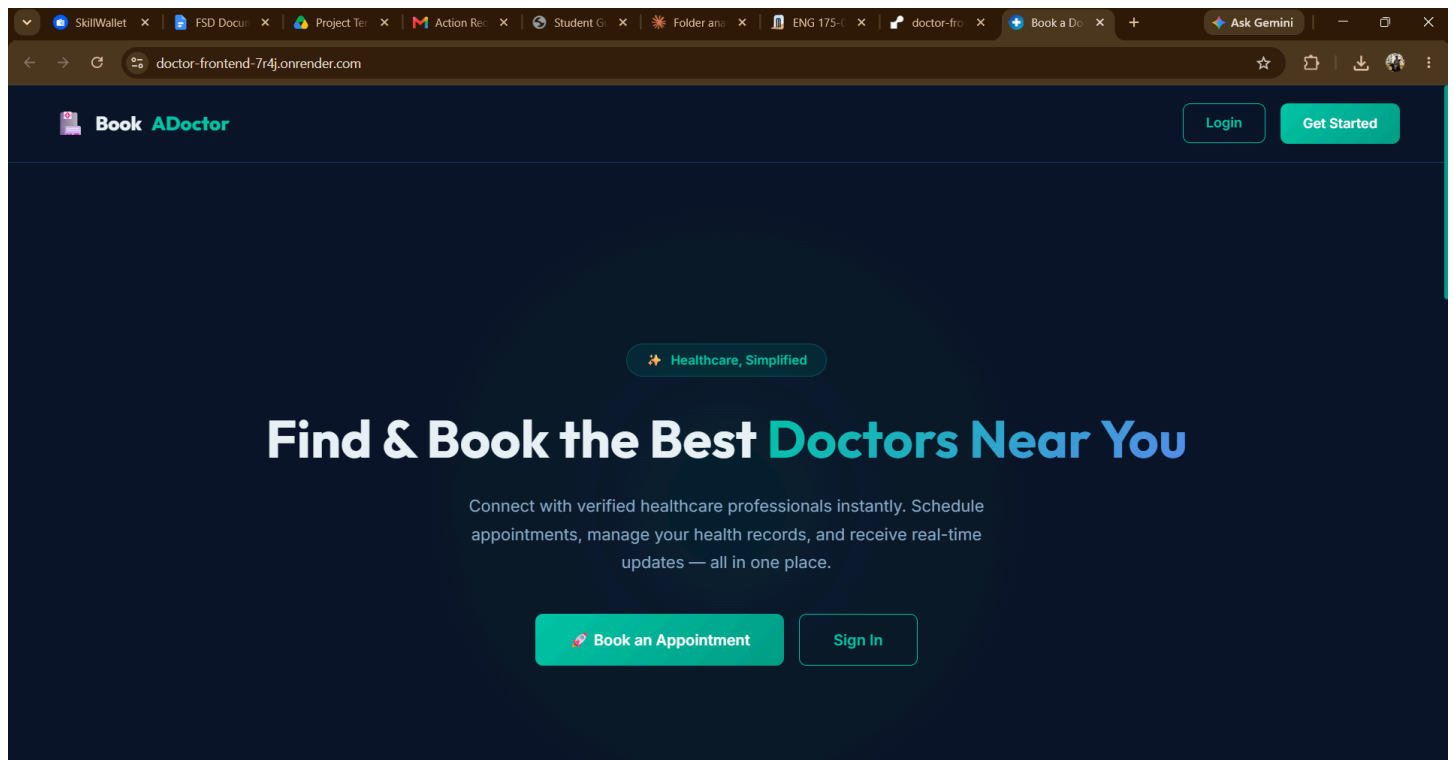
- Passwords are never returned in API responses — userController/adminController explicitly .select("-password") when fetching user documents.
- The JWT secret and MongoDB credentials must be kept private in server/.env and rotated immediately if ever exposed (see Setup Instructions § 4.2).
- Tokens are not currently blacklisted/revoked server-side on logout — "logging out" simply removes the token from localStorage on the client. A stolen token remains valid until it naturally expires (JWT_EXPIRES_IN).

9. User Interface

Screenshots of the deployed application (doctor-frontend-7r4j.onrender.com), showing the main flows across all three roles.

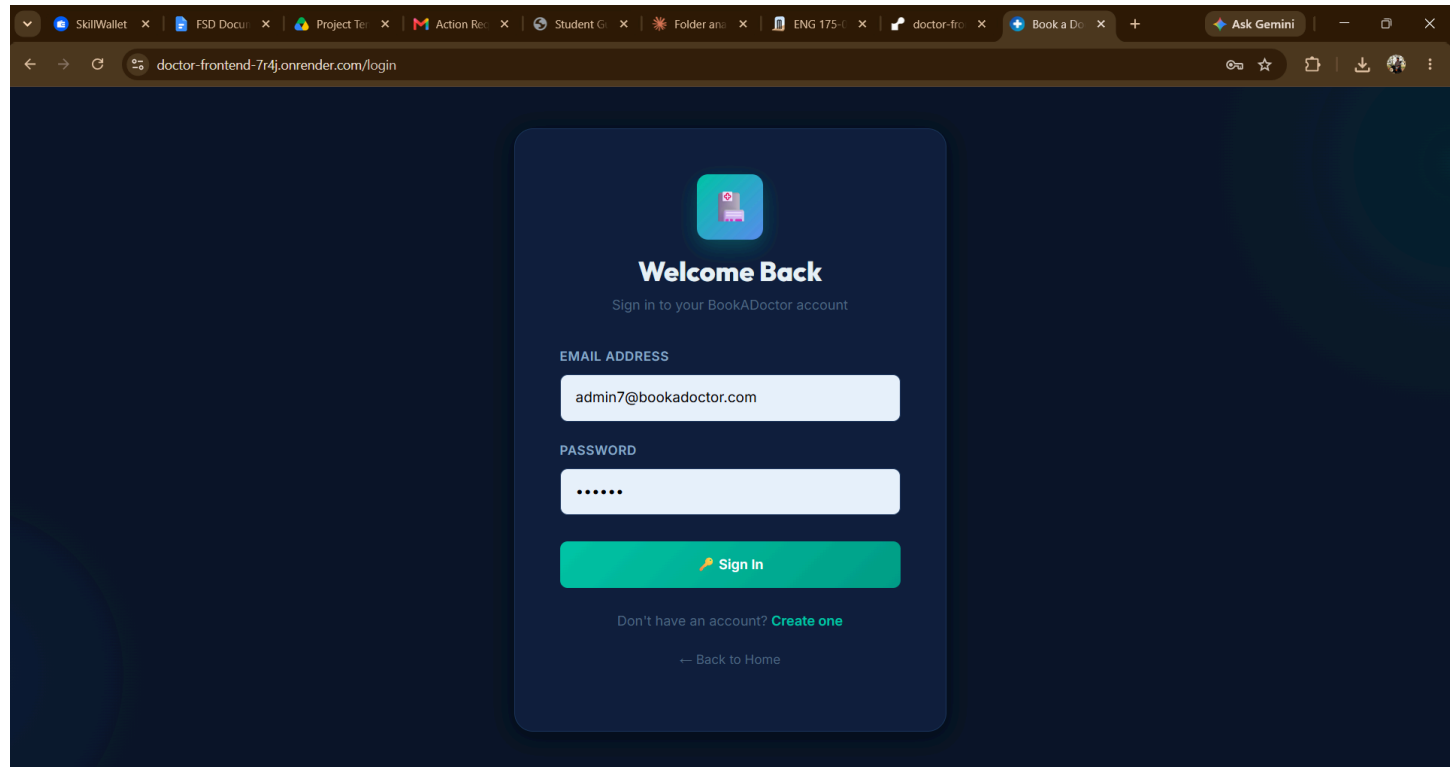
9.1 Landing Page

The public homepage introduces the platform with a "Healthcare, Simplified" tagline and calls to action to book an appointment or sign in.



9.2 Login

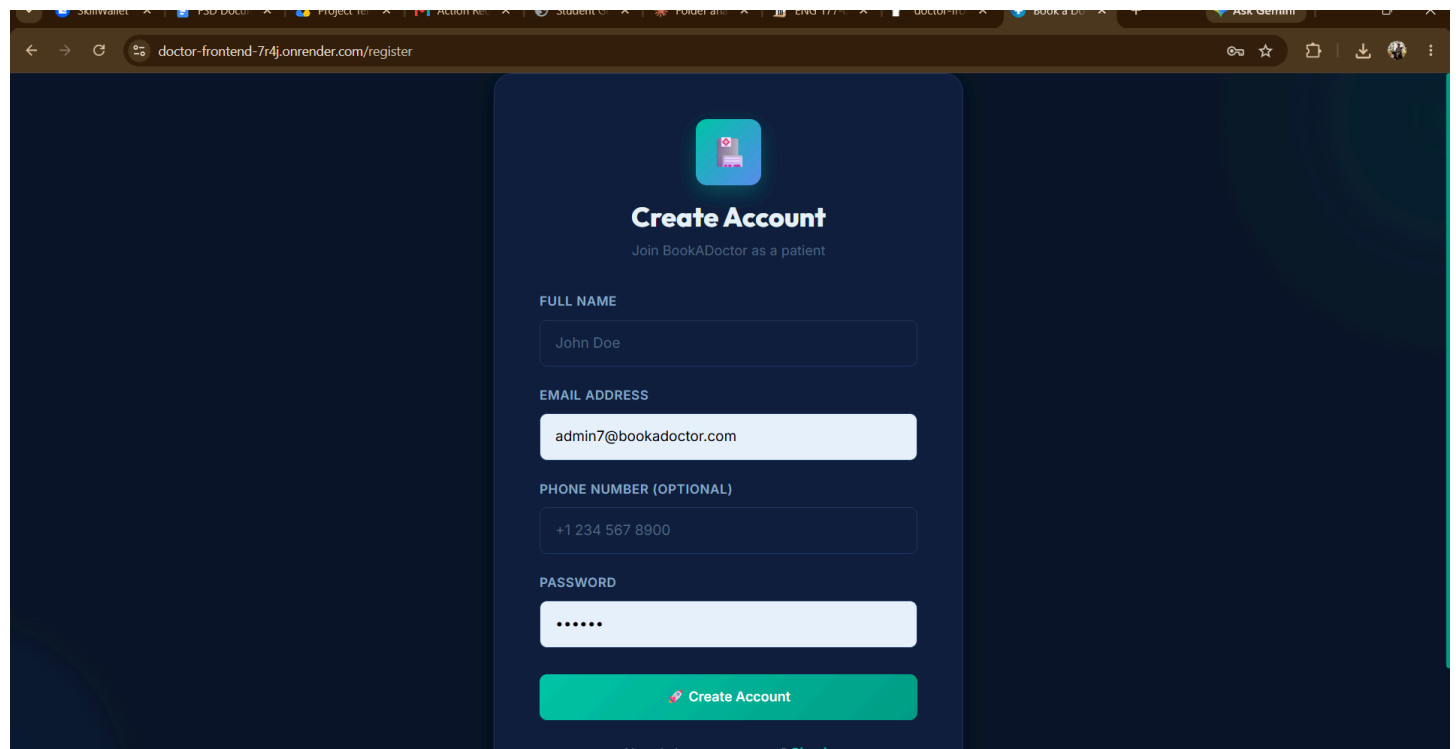
Existing users sign in with their email and password to receive a JWT session.



The screenshot shows a web browser window with the URL `doctor-frontend-7r4j.onrender.com/login`. The page features a dark blue background with a central white login card. At the top of the card is a circular logo with a red cross and a blue background. Below the logo, the text "Welcome Back" is displayed in a bold, dark blue font, followed by the subtitle "Sign in to your BookADoctor account" in a smaller, lighter blue font. The login form consists of two input fields: "EMAIL ADDRESS" with the value `admin7@bookadoctor.com` and "PASSWORD" with masked characters. Below these fields is a green "Sign In" button with a white arrow icon. At the bottom of the card, there is a link "Don't have an account? [Create one](#)" and a link "← Back to Home".

9.3 Registration

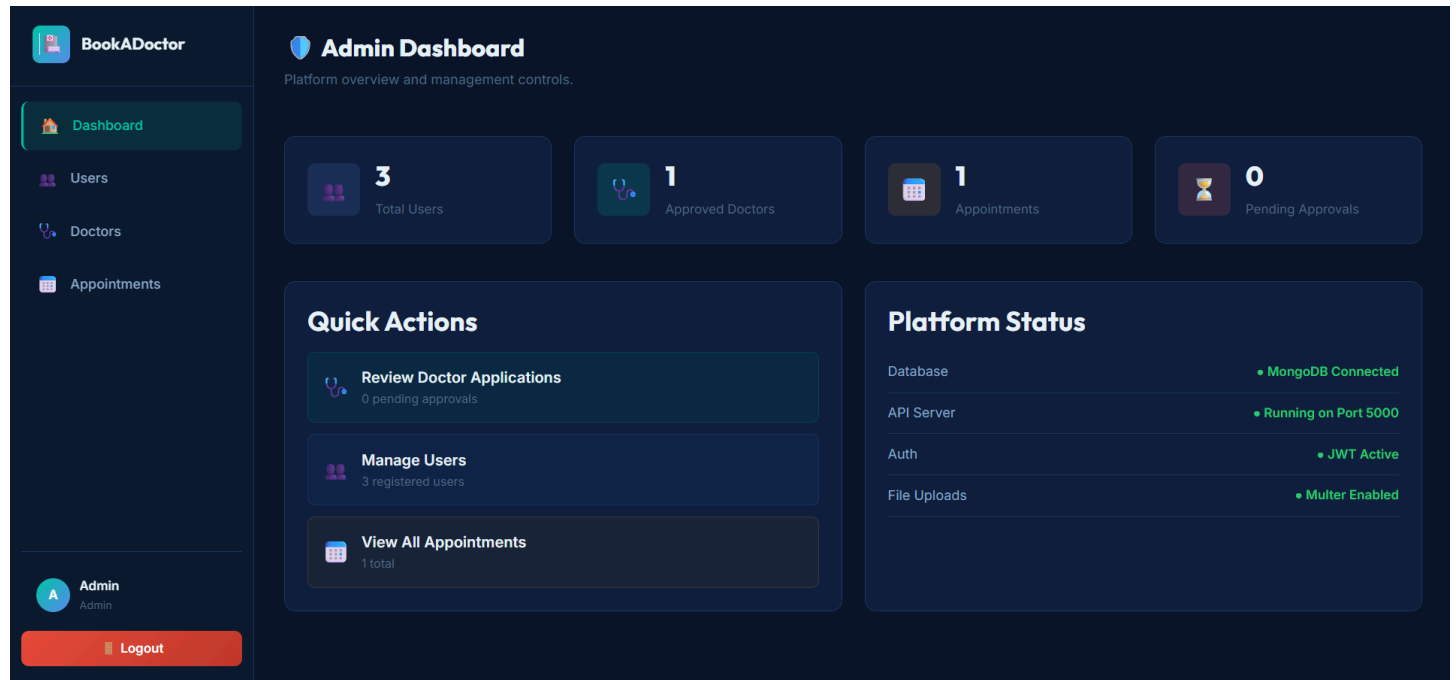
New patients create an account by providing their name, email, optional phone number, and password.



The screenshot shows a web browser window with the URL `doctor-frontend-7r4j.onrender.com/register`. The page features a dark blue background with a central white registration card. At the top of the card is a circular logo with a red cross and a blue background. Below the logo, the text "Create Account" is displayed in a bold, dark blue font, followed by the subtitle "Join BookADoctor as a patient" in a smaller, lighter blue font. The registration form consists of four input fields: "FULL NAME" with the value `John Doe`, "EMAIL ADDRESS" with the value `admin7@bookadoctor.com`, "PHONE NUMBER (OPTIONAL)" with the value `+1 234 567 8900`, and "PASSWORD" with masked characters. Below these fields is a green "Create Account" button with a white arrow icon. At the bottom of the card, there is a link "Already have an account? [Sign in](#)".

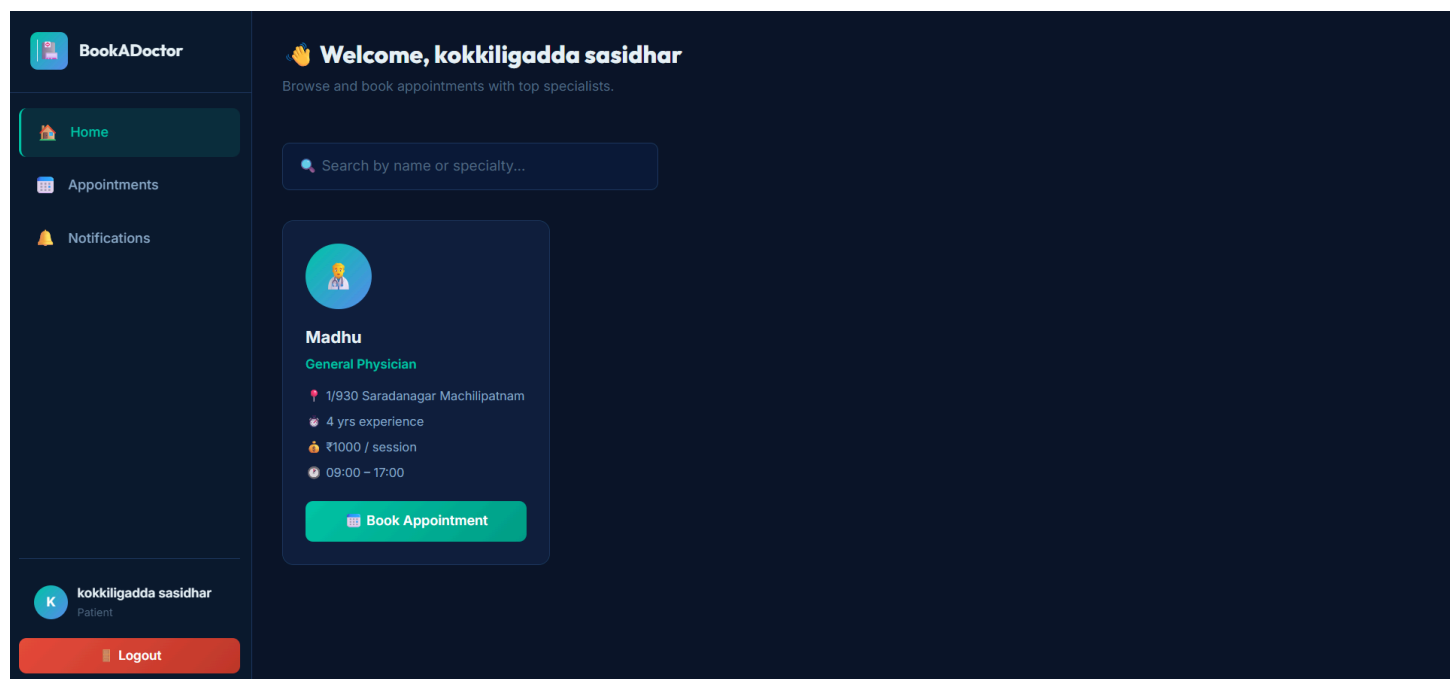
9.4 Admin Dashboard

Admins get a platform-wide overview — total users, approved doctors, appointments, and pending approvals — along with quick actions and live platform/service status.



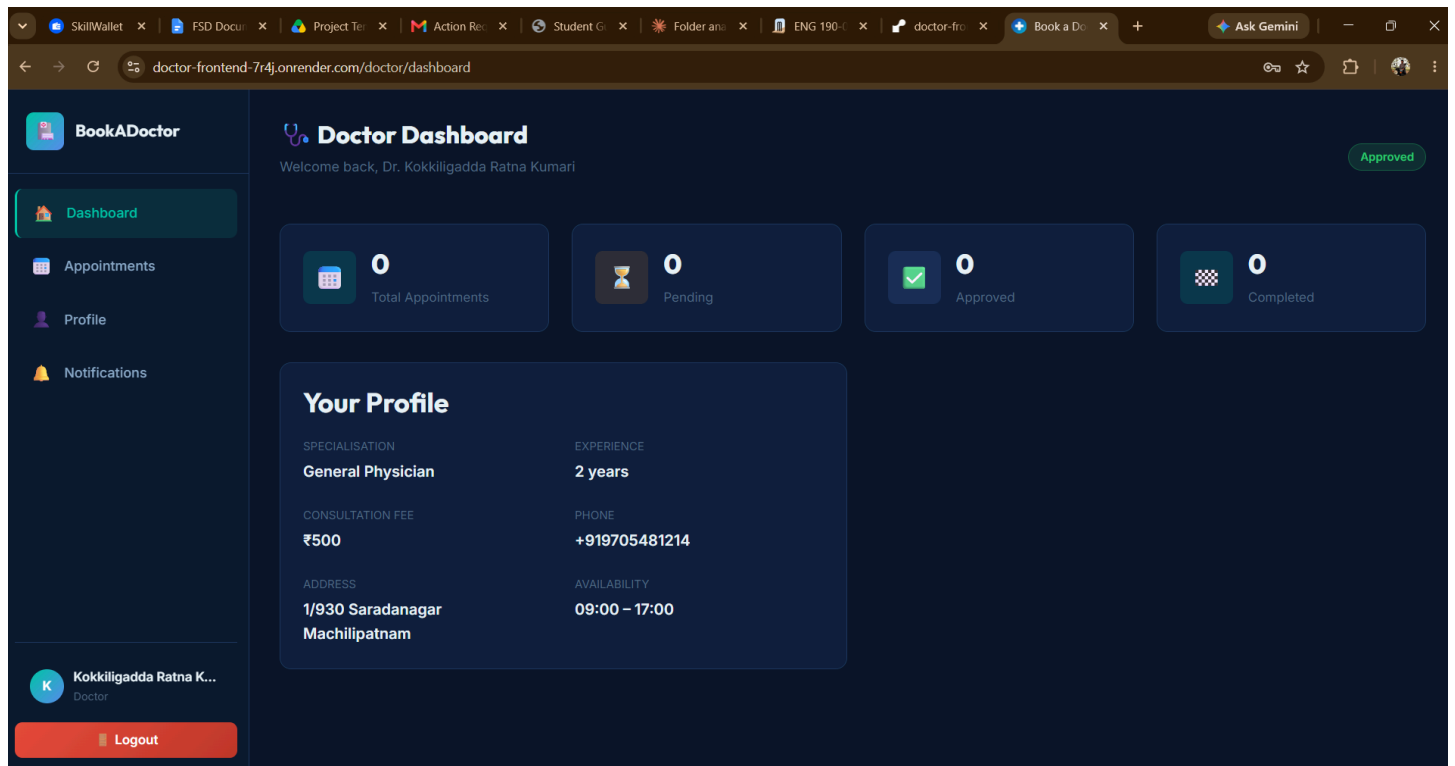
9.5 Patient Dashboard

Patients land on a searchable list of approved doctors, with each doctor card showing specialisation, location, experience, fee, and availability, plus a one-click "Book Appointment" action.



9.6 Doctor Dashboard

Approved doctors see their appointment stats (total, pending, approved, completed) and a summary of their public profile (specialisation, fee, phone, address, availability).



10. Testing

This project does not currently include an automated test suite (no test scripts or *.test.js/*.spec.js files are present in either client/ or server/). Testing was carried out manually, using the following approach and tools:

10.1 Local Development Testing (VS Code)

- The client and server were run locally side-by-side in **VS Code**, using its integrated terminal to run `npm run dev` (backend, via nodemon) and `npm run dev` (frontend, via Vite).
- VS Code's built-in debugging and console output were used to trace request/response flow and catch runtime errors during development.
- Each API endpoint was manually exercised through the running frontend (registering, logging in, applying as a doctor, booking/approving/cancelling appointments, admin actions) to confirm expected behavior end-to-end before moving on to the next feature.
- Edge cases were checked manually per flow — e.g. double-booking the same doctor/date/time slot, submitting a doctor application twice, accessing an admin route as a non-admin user, and hitting protected routes without a token — verifying the correct error responses (400, 401, 403, 404) documented in Section 7.

10.2 Deployment Testing (Render)

- Both the frontend and backend are deployed on **Render**:
 - Backend: doctor-backend-l8lk.onrender.com
 - Frontend: doctor-frontend-7r4j.onrender.com
- After each deployment, the same manual test flows (registration → login → doctor application → admin approval → booking → status updates) were re-run against the live Render URLs to confirm the deployed build behaves the same as local, including CORS behavior between the two separate Render services (validated by the allowedOrigins regex for *.onrender.com in server.js) and correct environment variable resolution (VITE_API_BASE_URL pointing at the live backend).
- Render's deployment logs were used to catch build/runtime issues (e.g. missing environment variables, failed MongoDB connections) after each push.

10.3 Recommendations for Future Test Coverage

To harden the project going forward, consider adding:

- **Backend unit/integration tests** (e.g. Jest + Supertest) for controllers and routes, especially auth and role-based access checks.
- **Frontend component tests** (e.g. React Testing Library) for critical flows like login, booking, and protected routing.
- **CI pipeline** to run tests automatically on each push before Render deployment.

11. Screenshots or Demo

Live Demo: <https://doctor-frontend-7r4j.onrender.com>

Note: this is deployed on Render's free tier, so the first request after a period of inactivity may take a moment to load while the service spins back up.

For screenshots of each major screen (landing page, login, registration, admin dashboard, patient dashboard, doctor dashboard), see Section 9 — User Interface.

12. Known Issues

The following bugs, gaps, and limitations should be understood by anyone using or extending this project:

12.1 Testing & Reliability

- **No automated test suite** — the project has no unit, integration, or end-to-end tests (see Section 10), so regressions can only be caught through manual testing.

- **No CI/CD pipeline** — deployments to Render happen without any automated build/test gate.

12.2 Security & Auth

- **No token revocation** — JWTs remain valid until they expire (JWT_EXPIRES_IN, default 7 days); logging out only clears the token client-side, so a stolen/leaked token can't be invalidated early.
- **No password reset / "forgot password" flow** — users who forget their password have no self-service way to recover their account.
- **No email verification** — accounts are usable immediately after registration with no email confirmation step.
- **Broad CORS policy** — the backend allows any *.onrender.com subdomain via regex, which is wider than strictly necessary and could allow other Render-hosted apps to make credentialed requests.
- **File upload validation is extension-based only** — multer's fileFilter checks the file extension (e.g. .pdf, .jpg) but not the actual file content/MIME type, so a malicious file renamed with an allowed extension could bypass the check.
- **Sensitive credentials shipped in the repo** — the version of this project reviewed had a real MongoDB Atlas connection string, JWT secret, and admin password committed in server/.env (see Section 4.2's security note). These must be rotated and never committed going forward.

12.3 Data Modeling

- **userId / doctorId fields are plain strings, not Mongoose ObjectId references** — relationships between User, Doctor, and Appointment are resolved manually in controllers rather than via .populate(), which is more error-prone and doesn't benefit from Mongoose's built-in referential validation.
- **Appointment date is stored as a string (DD-MM-YYYY)** rather than a native Date type — this makes chronological sorting, timezone handling, and date-range queries harder and more error-prone than they would be with a proper Date field.
- **No uniqueness constraint at the schema level for appointment slots** — double-booking prevention is enforced only in appointmentController.bookAppointment via a query check, not a database-level unique index, so it's theoretically possible for two near-simultaneous requests to both pass the check (race condition) and create conflicting bookings.

12.4 File Storage

- **Uploaded documents are stored on local disk (server/uploads/)** — on Render's free tier (and most PaaS hosts), the filesystem is ephemeral, so uploaded appointment documents can be lost whenever the service restarts or redeploys. A persistent storage solution (e.g. S3, Cloudinary) would be needed for production reliability.

12.5 Scalability / API Design

- **No pagination** on list endpoints (getAllUsers, getAllDoctors, getAllAppointments, etc.) — all matching documents are returned in a single response, which will not scale well as data grows.
- **No rate limiting** on any endpoint, including login and register, leaving them open to brute-force or spam attempts.
- **No centralized input validation library** (e.g. Joi/Zod/express-validator) — validation relies on Mongoose schema constraints and ad hoc checks in controllers, so some invalid input may only be caught late or inconsistently across endpoints.

12.6 Hosting

- **Cold starts on Render's free tier** — both frontend and backend may take extra time to respond after a period of inactivity, which can look like a bug (slow/failed initial load) to first-time users of the demo link.

13. Future Enhancements

Potential improvements that would extend the platform beyond its current scope:

13.1 Core Features

- **Online payments** — integrate a payment gateway (e.g. Razorpay, Stripe) so patients can pay the consultation fee at booking time instead of paying offline.
- **Video/telemedicine consultations** — add real-time video calling (e.g. WebRTC, Twilio) for remote appointments.
- **Real-time notifications** — replace the current stored-notification model with real-time push updates (e.g. WebSockets/Socket.IO) so users see new appointment requests or status changes instantly, without needing to refresh.
- **Doctor ratings & reviews** — let patients rate and review doctors after a completed appointment, and surface average ratings on doctor cards.
- **Search & filtering improvements** — filter doctors by specialisation, fee range, availability, or location, in addition to the existing name/specialty search.
- **Calendar-based slot picker** — replace manual date/time entry with a visual calendar showing a doctor's real availability and already-booked slots.
- **Prescriptions & medical history** — allow doctors to attach a digital prescription or notes to a completed appointment, viewable later by the patient.

13.2 Account & Security

- **Password reset / "forgot password" flow** via email.
- **Email verification** on registration.

- **Two-factor authentication (2FA)** for admin and doctor accounts.
- **Token revocation / refresh-token strategy** — support logging out a specific device or invalidating a token before natural expiry.
- **Stronger file validation** — verify uploaded file content/MIME type, not just extension, and scan uploads for malware.

13.3 Data & Infrastructure

- **Migrate userId/doctorId fields to proper Mongoose ObjectId references** with .populate(), and store appointment date as a native Date type.
- **Database-level uniqueness constraint** (or an atomic transaction) for appointment slots, closing the race-condition gap noted in Known Issues.
- **Move file uploads to persistent cloud storage** (e.g. AWS S3, Cloudinary) so documents survive redeloys.
- **Pagination and filtering** on all list endpoints (getAllUsers, getAllDoctors, getAllAppointments).
- **Rate limiting** on public endpoints (login, register) to reduce abuse.
- **Centralized request validation** (e.g. Joi, Zod, or express-validator) applied consistently across all routes.

13.4 Testing & DevOps

- **Automated test suite** — unit tests for controllers/models (Jest), API integration tests (Supertest), and frontend component tests (React Testing Library).
- **CI/CD pipeline** to run tests and lint checks automatically before deploying to Render.
- **Structured logging & monitoring** (e.g. Winston/Pino + an error-tracking service like Sentry) to replace ad hoc console.error calls.

13.5 UX & Accessibility

- **Multi-language support (i18n)** for a broader patient base.
- **Accessibility audit** (WCAG compliance) — keyboard navigation, screen-reader labels, color contrast.
- **Mobile app** (React Native) sharing the existing backend API.
- **Admin analytics dashboard** — trends in appointment volume, popular specialisations, doctor performance, etc.